

Chapter Two

Structure

- ✓ A structure is a collection of one or more variable of different data types grouped together to be referred using a single name (group name) and a member name.
- ✓ You can refer to a structure as a single structure variable, and you also can initialize, read and change the parts of a structure (the individual variables that make it up).
- ✓ Each element (called a member variable) in a structure can be of different data type.
- ✓ Structures are used to represent a record/row information,
 - Suppose you want to keep track of your books in a library.
 - You might want to track the following attributes about each book:
 - Title
 - Author
 - Subject
 - Book ID

This information can be represented using structure as in *example 2 below*.

Defining a Structure

To define a structure, you must use the **struct statement** (using the struct keyword).

The **struct** statement defines a new data type, with more than one member, for your program. The format of the struct statement is this:

- ✓ The General Syntax of structure declaration is:

```
Struct <structure tag>
{
    Member definition;
    Member definition;
    ...
    Member definition;
}<one or more structure variables>;
```

Note:- structure declaration always ends with semicolon

- ✓ Let us see examples

Example1.

```
E.g.: struct Inventory
{
    char description[15];char
    part_no[6];
    int quantity;
    float cost;
}; //
```

Structure tag

member variables of the structure

Example2.

```
struct Books
{
    char title[50]; char
    author[50]; char
    subject[100];int
    book_id;
}book; // defining a structure variable like this is also another option.
```

✓ Another example 3

```
struct Student
{
    char ID[8]; char
    FName[15];char
    LName[15];char
    Sex;
    int age; float
    CGPA;
};
```

- ✓ The above “Student” structure is aimed to store student record/row information with all the relevant details.
- ✓ After the definition of the structure, one can declare a structure variable using the **structure tag**. If we need two variables to have the above structure property then the declaration would be:

```
struct Inventory inv1,inv2; //or
struct Student Stud1,stud2,Stud3;
```
- ✓ Structure tag is not a variable name. Unlike array names, which reference the array as variables, a structure tag is simply a label for the structure’s format.
- ✓ The structure tag Inventory informs C++ that the tag called Inventory looks like two character arrays followed by one integer and one float variables.
- ✓ A structure tag is actually a newly defined data type that you, the programmer, defined.

Note: A structure can be considered as a user defined data type.

Referencing members of a structure

- ✓ To refer to the members of a structure we need to use the **dot operator (.)**
- ✓ The General syntax to access members of a structure variable would be:

```
VarName.Member
```
- ✓ Where VarName is the variable name of the structure variable and *Member* is variable name of the members of the structure

Programming Fundamentals II

Eg:

For the above student structure:

```
struct Student Stud; /*declaring Stud to have the property of the Studentstructure*/
strcpy(Stud.FName,"Abebe"); //assigned Abebe as First Name
Stud.CGPA=3.21; //assignes 3.21 as CGPA value of Abebe
cout<<Stud.FName; //display the name
cout<<Stud.CGPA;      // display the CGPA of Abebe
```

Initializing Structure Data

- ✓ You can initialize members when you declare a structure, or you can initialize a structure in the body of the program. Here is a complete program.

```
struct cd_collection
{
    char title[25]; char
    artist[20];int
    num_songs; float
    price;
    char date_purchased[9];
} cd1 = {"Red Moon Men","Sams and the Sneeds", 12, 11.95,"08/13/93"};
```

```
cout<<"\n here is the info about cd1"<<endl;
cout<<cd1.title<<endl; cout<<cd1.artist<<endl;
cout<<cd1.num_songs<<endl; cout<<cd1.price<<endl;
cout<<cd1.date_purchased<<endl;
```

- ✓ A better approach to initialize structures is to use the dot operator(.). the dot operator is one way to initialize individual members of a structure variable in the body of your program. The syntax of the dot operator is :

structureVariableName.memberName

here is an example:

```
#include<iostream.>
using namespace std;
struct cd_collection
{
    char title[25]; char
    artist[20];int
    num_songs; float
    price;
    char date_purchased[9];
}cd1;
```

```
int main()
{
//initialize members here strcpy(cd1.title,"Red
    Moon Men");strcpy(cd1.artist,"Sams");
    cd1.num_songs= 12;
    cd1.price = 11.95f;
    strcpy(cd1.date_purchased,"22/12/02");
//print the data
    cout<<"\nHere is the info"<<endl; cout<<"Title :
    "<<cd1.title<<endl; cout<<"Artist :
    "<<cd1.artist<<endl; cout<<"Songs :
    "<<cd1.num_songs<<endl;cout<<"Price :
    "<<cd1.price<<endl;
    cout<<"Date purchased : "<<cd1.date_purchased;return 0;
}
```

Arrays of Structures

- ✓ Arrays of structures are good for storing a complete employee file, inventory file, or any other set of data that fits in the structure format.
- ✓ Consider the following structure declaration:

```
struct Company
{
    int employees;int
    registers; double
    sales;
}store[1000];
```

- ✓ In one quick declaration, this code creates 1,000 **store** structures with the definition of the **Company** structure, each one containing three members.
- ✓ NB. Be sure that your computer does not run out of memory when you create a large number of structures. Arrays of structures quickly consume valuable information.
- ✓ You can also define the array of structures after the declaration of the structure.

```
struct Company
{
    int employees;int
    registers; double
    sales;
}; // no structure variables defined yet
```

```
#include<iostream>
...
int main()
{
    struct Company store[1000]; //the variable store is array of the
    structure Company
    ...
}
```

Referencing the array structure

- ✓ The ***dot operator (.)*** works the same way for structure array element as it does for regular variables. If the number of employees for the fifth store (store[4]) increased by three, you could update the structure variable like this:

```
store[4].employees += 3;
```

Unlike in the case of arrays, where the whole content of an array could not be copied to another one using a simple statement, in structures, you can assign complete structures to one another by using array notation.

- ✓ To assign all the members of the 20th store to the 45th store, you would do this:

```
store[44] = store[19]; //copies all members from 20th store to 45thExample
```

```
// structure variables as array elements#include
<iostream>
using namespace std;
const int SIZE = 2; //number of parts in array struct part
//specify a structure name/label/tag
{
    char name[30];
    int modelnumber; //ID number of widget int
    partnumber; //ID number of widget partfloat cost;
    //cost of part
}p2= {"dgvc",45,67,78.0};

int main()
{
    part p1;
    p1=p2; int
    n;
    cout<<p1.name<<endl;
    part apart[SIZE]; //define array of structures for(n=0; n<SIZE; n++)
    //get values for all members
    {
        cout << endl;
        cout << "Enter model Name:";
```

```
        cin>>apart[n].name; /*read value from the keyboard using
array structure*/
        cout << "Enter model number:";
        cin >> apart[n].modelnumber; //get model numbercout <<
        "Enter part number:";
        cin >> apart[n].partnumber; //get part numbercout <<
        "Enter cost: ";
        cin >> apart[n].cost; //get cost
    }
    cout << endl;
    for(n=0; n<SIZE; n++) //show values for all members
    {
        cout <<" Name ," << apart[n].name;
        cout <<" Model ," << apart[n].modelnumber;cout <<
        "Part ," << apart[n].partnumber; cout << " Cost" <<
        apart[n].cost << endl;
    }
    return 0;
}
```

Structures With in Structures

We can nest structures within other structures. Nesting a structure means, defining a structure variable/s as a member variable of other structure as indicated in example below.
Example:

In the program we want to create a data structure that stores the dimensions of a typical room: its length and width. We will use two variables of type **Distance (structure)** as the length and width variables in Room (structure).

Here is the program,

```
#include <iostream>

using namespace std;

struct Distance //English distance
{
    int feet; float
    inches;
};

struct Room //rectangular area
{
    Distance length; //length of rectangleDistance
```

```
        width; //width of rectangle
    };

    int main()
    {
        Room dining; //define a room dining.length.feet = 13;
        //assign values to roomdining.length.inches = 6.5;
        dining.width.feet = 10;
        dining.width.inches = 0.0;
        //convert length & width
        float l = dining.length.feet + dining.length.inches/12;float w =
        dining.width.feet + dining.width.inches/12;
        //find area and display it
        cout << "Dining room area is " << l * w<< " square feet\n" ;return 0;
    }
```

Note: Nesting a structure can be used to implement real world problem.

Structures as Function Arguments

You can pass a structure as a function argument in very similar way as you pass any other **variable or pointer**. You would access structure variables in the similar way as you have accessed in the above example:

```
#include <iostream>
#include <cstring>
using namespace std;
void printBook( struct Books book );
struct Books
{
    char title[50];
    char author[50];
    char subject[100];
    int book_id;
};
int main( )
```

```
{
struct Books Book1; // Declare Book1 of type Book
struct Books Book2; // Declare Book2 of type Book
// book 1 specification
strcpy( Book1.title, "Learn C++ Programming");
strcpy( Book1.author, "Chand Miyan");
strcpy( Book1.subject, "C++ Programming");
Book1.book_id = 6495407;
// book 2 specification
strcpy( Book2.title, "Telecom Billing");
strcpy( Book2.author, "Yakit Singha");
strcpy( Book2.subject, "Telecom");
Book2.book_id = 6495700;
// Print Book1 info
printBook( Book1 );

// Print Book2 info
printBook( Book2 );
return 0;
}
void printBook( struct Books book )
{
cout << "Book title : " << book.title <<endl;
cout << "Book author : " << book.author <<endl;
cout << "Book subject : " << book.subject <<endl;
cout << "Book id : " << book.book_id <<endl;
}
```

When the above code is compiled and executed, it produces the following result:

```
Book title : Learn C++ Programming
Book author : Chand Miyan
Book subject : C++ Programming
Book id : 6495407
Book title : Telecom Billing
Book author : Yakut Singha
Book subject : Telecom
Book id : 6495700
```

Pointers to Structures

You can define pointers to structures in very similar way as you define pointer to any other variable as follows:

```
struct Books *struct_pointer;
```

Now, you can store the address of a structure variable in the above defined pointer variable. To find the address of a structure variable, place the '&' operator before the structure's name as follows:

```
struct_pointer = &Book1;
```

To access the members of a structure using a pointer to that structure, you must

Programming Fundamentals II

use the -> operator as follows:

```
struct_pointer->title;
```

Let us re-write above example using structure pointer, hope this will be easy for you to understand the concept:

```
#include <iostream>
#include <cstring>
using namespace std;
void printBook( struct Books *book );
struct Books
{
    char title[50];
    char author[50];
    char subject[100];
    int book_id;
};
int main( )
{
    struct Books Book1; // Declare Book1 of type Book
    struct Books Book2; // Declare Book2 of type Book
    // Book 1 specification
    strcpy( Book1.title, "Learn C++ Programming");
    strcpy( Book1.author, "Chand Miyan");
    strcpy( Book1.subject, "C++ Programming");
    Book1.book_id = 6495407;
    // Book 2 specification
    strcpy( Book2.title, "Telecom Billing");
    strcpy( Book2.author, "Yakit Singha");
    strcpy( Book2.subject, "Telecom");
    Book2.book_id = 6495700;
    // Print Book1 info, passing address of structure
    printBook( &Book1 );
    // Print Book2 info, passing address of structure
    printBook( &Book2 );
    return 0;
}
// This function accept pointer to structure as parameter.
void printBook( struct Books *book )
{
    cout << "Book title : " << book->title <<endl;
    cout << "Book author : " << book->author <<endl;
    cout << "Book subject : " << book->subject <<endl;
    cout << "Book id : " << book->book_id <<endl;
}
```

When the above code is compiled and executed, it produces the following result:

Book title : Learn C++ Programming

Book author : Chand Miyan

Programming Fundamentals II

Book subject : C++ Programming

Book id : 6495407

Book title : Telecom Billing

Book author : Yakut Singha

Book subject : Telecom

Book id : 6495700